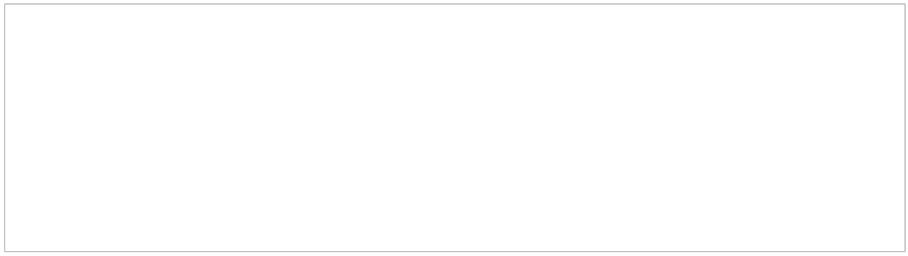

Interview question • LLM Scientist): Design + implement a "Regulatory QA Copilot" with Agentic RAG + MCP

Scenario:

You're building an internal assistant for agronomy/regulatory teams. It must answer questions about *product labels, safety datasheets, and internal regulatory memos*, and it must **never** invent citations. Some questions require multi-step reasoning ("compare" / "summarize changes" / "find the exact clause").

What you must build (in code/pseudocode)

Implement an **agentic RAG pipeline** with a minimal **Model Control Plane (MCP)** interface.

Inputs

- `question: str`
- `corpus: List[Document]` (text + metadata like doc_id, title, effective_date, section headings)
- `tools`:
  1. `retrieve(query) -> List[Chunk]` (BM25/embedding hybrid is a plus)
  2. `rerank(question, chunks) -> List[Chunk]`
  3. `extract_citations(chunks) -> List[Citation]` (doc_id + chunk span)
  4. `mcp.route(task) -> ModelSpec` (choose model + budget)
  5. `mcp.policy_check(draft, citations) -> Pass/Fail + reason`

Output

- `answer: str`

- `citations: List[Citation]`
- `trace: List[Step]` (what the agent decided and why)

Requirements (what the interviewer is testing)

1. **RAG correctness**
   - Must return answers **grounded in retrieved chunks**.
   - If evidence is insufficient, must say **"I don't have enough context"** and ask for the missing doc/section.
2. **Agentic behavior**
   - Decide whether to do:
     - single-shot retrieval
     - query rewrite + re-retrieve
     - multi-hop retrieval (find definition → then apply)
     - compare two sources (must cite both)
3. **MCP thinking**
   - Route cheap vs expensive models based on task difficulty and risk.
   - Enforce policies (no uncited claims, max tokens, safety checks).
4. **Evaluation hooks**
   - Add a small offline evaluation function: citation coverage + "answer supported" heuristic.

Starter data structures (you can hand candidates this)

python

```python
@dataclass
class Document:
    doc_id: str
    title: str
    effective_date: str
    text: str

@dataclass
class Chunk:
    doc_id: str
```

```
    chunk_id: str
    text: str
    score: float
    metadata: dict  # e.g., section, page, effective_date

@dataclass
class Citation:
    doc_id: str
    chunk_id: str

@dataclass
class Step:
    name: str
    rationale: str
```

The core prompt

Implement:

python

```
def answer_question(question: str, corpus: list[Document], mcp, tools) ->
    ...
```

Copy code

What a strong solution should do (high-level algorithm)

1. **Classify the task** (agent “planner” step)
   - Is it “lookup”, “definition”, “compare”, “summarize”, “procedure”, or “risk/high-stakes”?
2. **MCP route** Copy code
   - Cheap model for query rewriting + filtering.
   - Stronger model for final synthesis **only if** evidence is sufficient and question is complex.
3. **Retrieve → rerank**
   - Retrieve top-N, rerank to top-K.

4. **Grounding gate**
    - If top-K lacks coverage, agent rewrites query or multi-hops.
5. **Synthesize answer**
    - Answer using only the cited chunks.
6. **Policy check**
    - MCP rejects if answer contains claims not supported by citations.
7. **Return answer + citations + trace**

Concise concept explanations (as they appear in the solution)

GenAI

A **generative model** produces text by modeling the probability of the next token. In this system it's used for **planning (agent)** and **synthesis (final answer)**.

RAG

Retrieval-Augmented Generation: you first retrieve relevant evidence from a corpus, then generate an answer **conditioned on that evidence**, improving factuality and enabling citations.

Retriever vs Reranker

- **Retriever**: fast, broad recall (gets "maybe relevant" chunks).
- **Reranker**: slower, high precision (orders chunks by relevance to the question).

Query rewriting

An LLM reformulates the question into better search queries (adds synonyms, expands acronyms, injects missing context) to improve retrieval recall.

Multi-hop retrieval

Retrieve in steps: first find a **definition or entity**, then use it to retrieve the **specific rule/procedure** needed for the final answer.

Context window management

Only a limited amount of retrieved text fits into the model's input. You select the best chunks (top-K), deduplicate, and keep citations aligned to what the model saw.

Agentic AI

An **agent** is an LLM-driven controller that chooses actions (retrieve, rewrite, compare, stop) based on intermediate results, not just a single prompt-response.

MCP (Model Control Plane)

An orchestration layer that governs:

- **routing** (which model for which step),
- **budgets** (latency/cost),
- **policies** (grounding, safety, allowed tools),
- **telemetry** (trace/logging).

Scoring rubric (what you evaluate in the interview)

- **Correctness**: answer is fully supported by citations (or admits insufficiency).
- **Robustness**: handles missing/conflicting sources; avoids hallucinations.
- **Design clarity**: clean separation of planner/retrieval/synthesis/policy layers.
- **MCP realism**: explicit routing + policy gates + budgets.
- **Traceability**: produces a readable `trace` explaining decisions.

If you want, I can also provide:

- a **complete reference implementation** (Python, minimal dependencies) using a toy retriever + reranker, and
- a set of **test cases** (easy lookup, compare, insufficient evidence, conflicting docs)

give a complete reference implemenation

No — *this full agentic RAG + MCP system* is usually too big for a **single 60-minute live coding** interview **unless** they give a scaffold + toy corpus + stubbed tools. What's realistic in 1 hour:

- **Most likely:** implement a *grounded RAG answerer* (retrieve → rerank → synthesize → "insufficient evidence" gate) + clean data structures + a couple tests.
- **Maybe:** add a tiny "agent loop" (retry with query rewrite once) and a simple "MCP router" (if question is "compare" use stronger model).
- **Unlikely:** full multi-hop planning, robust policy enforcement, rich trace/eval *from scratch*.

Below is a **complete reference implementation** you can practice with. It's designed to be runnable **offline** with a toy corpus, and the "LLM" parts are implemented as deterministic heuristics so you can code it in an interview without external APIs. (In a real system, those pieces would call models.)

Complete reference implementation (single Python file)

python

```python
from __future__ import annotations

from dataclasses import dataclass
from typing import List, Tuple, Dict, Any, Optional
import re
import math
from collections import Counter, defaultdict


# ----------------------------
# Data Structures
# ----------------------------

@dataclass
class Document:
    doc_id: str
    title: str
    effective_date: str
    text: str
```

```python
@dataclass
class Chunk:
    doc_id: str
    chunk_id: str
    text: str
    score: float
    metadata: Dict[str, Any]


@dataclass
class Citation:
    doc_id: str
    chunk_id: str


@dataclass
class Step:
    name: str
    rationale: str


# ----------------------------
# Utility: tokenization + scoring
# ----------------------------

_WORD_RE = re.compile(r"[a-zA-Z0-9]+")


def tokenize(text: str) -> List[str]:
    return [t.lower() for t in _WORD_RE.findall(text)]


def cosine_sim(a: Counter, b: Counter) -> float:
    # Cosine similarity between sparse term-frequency vectors
    dot = 0.0
    for k, va in a.items():
        dot += va * b.get(k, 0)
    na = math.sqrt(sum(v * v for v in a.values()))
```

```
        nb = math.sqrt(sum(v * v for v in b.values()))
        if na == 0.0 or nb == 0.0:
            return 0.0
        return dot / (na * nb)


    def chunk_text(doc: Document, chunk_size: int = 250, overlap: int = 40) ->
        """
        Simple character-based chunker with overlap.
        In real systems you'd do token-based chunking + structure-aware bounda
        """
        text = doc.text.strip()
        chunks: List[Chunk] = []
        start = 0
        i = 0
        while start < len(text):
            end = min(len(text), start + chunk_size)
            chunk_str = text[start:end].strip()
            if chunk_str:
                chunks.append(
                    Chunk(
                        doc_id=doc.doc_id,
                        chunk_id=f"{doc.doc_id}:{i}",
                        text=chunk_str,
                        score=0.0,
                        metadata={
                            "title": doc.title,
                            "effective_date": doc.effective_date,
                            "start_char": start,
                            "end_char": end,
                        },
                    )
                )
            i += 1
            if end == len(text):
                break
            start = max(0, end - overlap)
        return chunks
```

```
# ----------------------------
# Toy Retriever + Reranker tools
# ----------------------------

class Tools:
    """
    Tools container: retrieve, rerank, extract_citations.
    Implemented as offline heuristics for interview practice.
    """
    def __init__(self, corpus: List[Document], chunk_size: int = 260, over
        self.documents = corpus
        self.chunks: List[Chunk] = []
        for d in corpus:
            self.chunks.extend(chunk_text(d, chunk_size=chunk_size, overla

        # Precompute TF vectors for retrieval
        self.chunk_tf: Dict[str, Counter] = {}
        for ch in self.chunks:
            self.chunk_tf[ch.chunk_id] = Counter(tokenize(ch.text))

    def retrieve(self, query: str, top_n: int = 12) -> List[Chunk]:
        """
        Recall-oriented retrieval: cosine similarity on TF vectors.
        (Stands in for BM25 / embedding retrieval.)
        """
        q_tf = Counter(tokenize(query))
        scored: List[Chunk] = []
        for ch in self.chunks:
            score = cosine_sim(q_tf, self.chunk_tf[ch.chunk_id])
            if score > 0.0:
                scored.append(Chunk(**{**ch.__dict__, "score": score}))
        scored.sort(key=lambda c: c.score, reverse=True)
        return scored[:top_n]

    def rerank(self, question: str, chunks: List[Chunk], top_k: int = 5) ->
        """
        Precision-oriented reranker: boosts exact phrase matches + rare-te
        """
        q_tokens = tokenize(question)
        q_tf = Counter(q_tokens)
```

```
            # rarity approximation
            df = defaultdict(int)
            for tok in set(q_tokens):
                for ch in chunks:
                    if tok in tokenize(ch.text):
                        df[tok] += 1

            reranked: List[Chunk] = []
            for ch in chunks:
                base = ch.score
                text_lower = ch.text.lower()

                phrase_boost = 0.0
                # boost 2-grams present verbatim
                for i in range(len(q_tokens) - 1):
                    bigram = f"{q_tokens[i]} {q_tokens[i+1]}"
                    if bigram in text_lower:
                        phrase_boost += 0.08

                rarity_boost = 0.0
                ch_tokens = set(tokenize(ch.text))
                for tok, qt in q_tf.items():
                    if tok in ch_tokens:
                        # fewer chunks contain tok => higher boost
                        rarity = 1.0 / (1 + df.get(tok, 0))
                        rarity_boost += 0.03 * rarity * qt

                new_score = base + phrase_boost + rarity_boost
                reranked.append(Chunk(**{**ch.__dict__, "score": new_score}))

            reranked.sort(key=lambda c: c.score, reverse=True)
            return reranked[:top_k]

        def extract_citations(self, chunks: List[Chunk]) -> List[Citation]:
            return [Citation(doc_id=c.doc_id, chunk_id=c.chunk_id) for c in chu


    # ----------------------------
    # Minimal MCP (Model Control Plane)
    # ----------------------------
```

```
@dataclass
class ModelSpec:
    name: str
    max_input_chars: int
    max_output_chars: int
    temperature: float
    rationale: str


class MCP:
    """
    Minimal MCP:
    - route(task): pick model spec
    - policy_check(draft, cited_chunks): ensure claims appear in cited tex
    """
    def route(self, task: str, risk: str) -> ModelSpec:
        # In real MCP: route to different deployed models + budgets.
        if risk == "high" or task in ("compare", "procedure"):
            return ModelSpec(
                name="strong-model",
                max_input_chars=2500,
                max_output_chars=900,
                temperature=0.0,
                rationale="High-stakes or complex task: use strongest mode
            )
        return ModelSpec(
            name="fast-model",
            max_input_chars=1800,
            max_output_chars=700,
            temperature=0.0,
            rationale="Low/medium risk: use fast model to reduce latency/c
        )

    def policy_check(self, answer: str, cited_chunks: List[Chunk]) -> Tupl
        """
        Grounding heuristic:
        - Require answer content words to substantially overlap with citat
        - Flag if answer introduces many tokens not present in citations.
        Real systems use entailment checks / attribution models / structur
```

```
        """
        cited_text = " ".join(ch.text.lower() for ch in cited_chunks)
        ans_tokens = [t for t in tokenize(answer) if len(t) > 3]
        if not ans_tokens:
            return False, "Empty answer."

        missing = 0
        for t in ans_tokens:
            if t not in cited_text:
                missing += 1

        missing_ratio = missing / max(1, len(ans_tokens))
        if missing_ratio > 0.35:
            return False, f"Answer appears to contain too many uncited ter
        return True, "Pass"


# ----------------------------
# "LLM-like" pieces (deterministic for offline interview practice)
# ----------------------------

def classify_task(question: str) -> Tuple[str, str]:
    """
    Returns (task_type, risk_level).
    """
    q = question.lower()
    if any(w in q for w in ["compare", "difference", "vs", "changed", "cha
        return "compare", "high"
    if any(w in q for w in ["how do i", "procedure", "steps", "must", "req
        return "procedure", "high"
    if any(w in q for w in ["define", "definition", "what does", "meaning"
        return "definition", "medium"
    return "lookup", "medium"


def rewrite_query(question: str) -> str:
    """
    Deterministic "query rewrite":
    - expand common abbreviations
    - add "effective date" when asking about changes
```

```
    """
    q = question
    q = re.sub(r"\bsds\b", "safety data sheet", q, flags=re.I)
    q = re.sub(r"\blabel\b", "product label", q, flags=re.I)
    if re.search(r"\b(compare|changed|change|difference|vs)\b", q, flags=r
        q += " effective date section clause"
    return q


def select_context(chunks: List[Chunk], max_chars: int) -> List[Chunk]:
    """
    Simple context window management: keep top chunks until budget.
    """
    out: List[Chunk] = []
    total = 0
    seen = set()
    for ch in chunks:
        if ch.chunk_id in seen:
            continue
        if total + len(ch.text) > max_chars:
            continue
        out.append(ch)
        seen.add(ch.chunk_id)
        total += len(ch.text)
    return out


def synthesize_answer(question: str, chunks: List[Chunk]) -> str:
    """
    Deterministic synthesizer:
    - For demo: stitch top evidence with light formatting.
    Real system: call LLM with constrained prompt to only use provided con
    """
    if not chunks:
        return "I don’t have enough context to answer from the available d

    lines = []
    lines.append("Answer (grounded in retrieved sources):")
    # naive: pick 2-3 best chunks
    for ch in chunks[:3]:
```

```
            title = ch.metadata.get("title", "")
            eff = ch.metadata.get("effective_date", "")
            excerpt = ch.text.strip().replace("\n", " ")
            if len(excerpt) > 240:
                excerpt = excerpt[:240].rstrip() + "…"
            lines.append(f"- [{title} | {eff}] {excerpt}")
        lines.append("")
        lines.append("If you want a tighter, single-paragraph answer, tell me t
        return "\n".join(lines)


    # ----------------------------
    # Main pipeline: Agentic RAG + MCP
    # ----------------------------

    def answer_question(
        question: str,
        corpus: List[Document],
        mcp: MCP,
        tools: Tools,
    ) -> Tuple[str, List[Citation], List[Step]]:
        trace: List[Step] = []

        task, risk = classify_task(question)
        trace.append(Step("classify_task", f"task={task}, risk={risk}"))

        model = mcp.route(task=task, risk=risk)
        trace.append(Step("mcp.route", f"{model.name}: {model.rationale}"))

        # First pass retrieval
        retrieved = tools.retrieve(question, top_n=12)
        trace.append(Step("retrieve", f"Retrieved {len(retrieved)} chunks for :

        reranked = tools.rerank(question, retrieved, top_k=6)
        trace.append(Step("rerank", f"Reranked to top {len(reranked)} chunks."

        context = select_context(reranked, max_chars=model.max_input_chars)
        trace.append(Step("context_select", f"Selected {len(context)} chunks w:

        # Agentic retry (one-step): if context seems weak, rewrite query and r
```

```
        if len(context) < 2:
            rq = rewrite_query(question)
            trace.append(Step("query_rewrite", f"Context weak; rewrote query t

            retrieved2 = tools.retrieve(rq, top_n=12)
            trace.append(Step("retrieve_retry", f"Retrieved {len(retrieved2)} 

            reranked2 = tools.rerank(question, retrieved2, top_k=6)
            trace.append(Step("rerank_retry", f"Reranked to top {len(reranked2

            context2 = select_context(reranked2, max_chars=model.max_input_cha
            trace.append(Step("context_select_retry", f"Selected {len(context2

            if len(context2) > len(context):
                context = context2

        # If still insufficient evidence, refuse safely
        if not context:
            trace.append(Step("insufficient_evidence", "No relevant evidence f
            return (
                "I don’t have enough context in the provided documents to answ
                "Please provide the relevant label/SDS/memo or specify the pro
                [],
                trace,
            )

        # Synthesize (deterministic demo)
        draft = synthesize_answer(question, context)
        trace.append(Step("synthesize", "Generated grounded draft from selecte

        # Policy check (grounding gate)
        ok, reason = mcp.policy_check(draft, context)
        trace.append(Step("mcp.policy_check", reason))

        if not ok:
            # Conservative fallback
            trace.append(Step("fallback", "Draft failed grounding check; retur
            return (
                "I found potentially relevant sources, but I can’t produce a r
                "Please narrow the question (product, region, effective date) 
```

```
                tools.extract_citations(context),
                trace,
            )

        citations = tools.extract_citations(context)
        return draft, citations, trace


# ----------------------------
# Simple evaluation hooks
# ----------------------------

def eval_citation_coverage(answer: str, cited_chunks: List[Chunk]) -> Dict
    """
    Heuristic metric: how much of answer vocabulary appears in citations.
    """
    cited_text = " ".join(ch.text.lower() for ch in cited_chunks)
    ans_tokens = [t for t in tokenize(answer) if len(t) > 3]
    if not ans_tokens:
        return {"coverage": 0.0, "missing_terms": []}

    missing = sorted({t for t in ans_tokens if t not in cited_text})
    coverage = 1.0 - (len(missing) / max(1, len(set(ans_tokens))))
    return {"coverage": coverage, "missing_terms": missing[:12]}


# ----------------------------
# Demo / Tests
# ----------------------------

def _toy_corpus() -> List[Document]:
    return [
        Document(
            doc_id="memo-1",
            title="Regulatory Memo: Herbicide X - Label Update",
            effective_date="2025-06-01",
            text=(
                "Label Update Summary:\n"
                "1) The maximum application rate for Herbicide X is 2.0 L/h
                "2) Buffer zone requirement: maintain 10 meters from surfa
```

```
                    "3) PPE: chemical-resistant gloves are required during mix
                    "Rationale: reduce aquatic exposure risk.\n"
                ),
            ),
            Document(
                doc_id="sds-1",
                title="Safety Data Sheet: Herbicide X",
                effective_date="2024-10-15",
                text=(
                    "Section 8: Exposure Controls / Personal Protection\n"
                    "Wear protective gloves and eye protection. Wash hands aft
                    "Section 2: Hazards Identification\n"
                    "May cause skin irritation. Avoid release to the environme
                ),
            ),
            Document(
                doc_id="label-1",
                title="Product Label: Herbicide X (US)",
                effective_date="2025-06-01",
                text=(
                    "Directions for Use:\n"
                    "Do not apply more than 2.0 L/ha per application.\n"
                    "Environmental Hazards:\n"
                    "Do not apply directly to water. Maintain a 10 m buffer fr
                    "Personal Protective Equipment:\n"
                    "Wear chemical-resistant gloves during mixing/loading.\n"
                ),
            ),
        ]


    def main():
        corpus = _toy_corpus()
        tools = Tools(corpus)
        mcp = MCP()

        q1 = "What is the maximum application rate for Herbicide X and what bu
        ans, cites, trace = answer_question(q1, corpus, mcp, tools)

        print("QUESTION:", q1)
```

```
    print("\n" + ans)
    print("\nCITATIONS:", cites)
    print("\nTRACE:")
    for s in trace:
        print(f"- {s.name}: {s.rationale}")

    # Optional eval
    # Reconstruct cited chunks for eval
    cited_chunk_map = {ch.chunk_id: ch for ch in tools.chunks}
    cited_chunks = [cited_chunk_map[c.chunk_id] for c in cites if c.chunk_
    metrics = eval_citation_coverage(ans, cited_chunks)
    print("\nEVAL:", metrics)


if __name__ == "__main__":
    main()
```

How to use this for interview practice (what to implement in 60 min)

If you want to practice for a realistic 1-hour session, aim to implement only:

1. `chunk_text`
2. `retrieve` (even naive keyword match is fine)
3. `rerank` (a simple overlap score)
4. `answer_question` with:
   - insufficient-evidence refusal
   - citations
   - **one** agent retry (query rewrite once)
5. 1–2 tests in `main()`

That's very doable live.

If you want, I can also rewrite this into a **more interview-friendly** version (shorter, fewer classes) while preserving the same concepts.

i have some questions first, give very short answers, what is tf vector

TF vector (Term Frequency vector):
A numeric vector where **each dimension is a word**, and the value is **how many times that word appears in a document (or chunk)**.

In short: *it's a bag-of-words count representation of text.*

benefits /purpose of having function like with parameters mentioned: e.g. route(task=task, risk=risk)

Very short, interview-style answer 👇

Purpose / benefits of `route(task=task, risk=risk)`

- **Explicit control**: makes *why* a model is chosen visible (task type + risk).
- **Separation of concerns**: business logic decides *what* the task is; MCP decides *how* to run it.
- **Cost & latency optimization**: route cheap models for low-risk tasks, strong models for high-risk ones.
- **Safety enforcement**: high-risk tasks automatically get stricter models / policies.
- **Scalability**: easy to add new tasks, risks, or models without touching core logic.
- **Auditability**: logs can show *which model was used and why* (important in regulated domains).

One-liner:
It's a clean control knob for **model selection, safety, and cost tradeoffs** based on task intent and risk.

no, why not route (task, risk)

Short, precise, interview-ready 👇

Why `route(task=task, risk=risk)` instead of `route(task, risk)`

- **Self-documenting**: named arguments make the call readable *without* checking the function signature.
- **Safer APIs**: prevents bugs from swapped arguments (`route(risk, task)` won't silently work).
- **Future-proofing**: you can add new parameters (e.g. `latency_budget=…`) without breaking callers.
- **Clarity in orchestration code**: MCP logic is control-plane code—explicitness matters more than brevity.
- **Partial overrides**: allows calls like `route(task=task)` while defaulting `risk` .

Interview one-liner:
Named parameters make MCP routing **explicit, safe, and extensible**, which is exactly what you want in control-plane code.

the order has to be preserved, e.g. route(risk = .., task =...)
Not

Ah — good catch. This is a **Python semantics point**, not an MCP/design one. Short answer:

In Python, order is *not* preserved for keyword arguments.

So both of these are **equivalent and valid**:

python

```
route(task=task, risk=risk)
route(risk=risk, task=task)
```

Copy code

Why this is actually a *feature*

- Keyword arguments are matched **by name, not position**
- Python internally maps them like a dictionary
- That's why reordering them **cannot change behavior**

What *does* require order

python

```
route(task, risk)        # positional → order matters
```

Why control-plane code prefers keywords anyway

Copy code

- Order independence = fewer bugs
- Call-site clarity matters more than call brevity
- You can safely refactor function signatures later

Interview one-liner

> *Keyword arguments intentionally ignore order; this guarantees correctness by binding parameters by meaning, not position.*

If you want, next I can explain **why positional args are risky specifically in MCP / agent orchestration code** — that's a strong interviewer follow-up answer.

give me a short answer: what do you mean by "Real system: call LLM with constrained prompt to only use provided context."

Short answer:

It means the LLM is **prompted and constrained** so it can **only generate answers from**

the retrieved context, not from its general knowledge.

In practice:

- The prompt includes *only* the selected chunks.
- The instructions explicitly say *"Answer using only this context. If missing, say you don't know."*
- This reduces hallucination and enforces grounding (RAG correctness).

give a small example of the prompt

Here's a **minimal, realistic constrained RAG prompt** 👇

vbnet

```
You are a regulatory QA assistant.

You must answer the question using ONLY the context provided below.
- Do NOT use prior knowledge.
- If the context does not contain enough information, say:
  "I don't have enough context to answer this question."

Question:
What is the maximum application rate for Herbicide X?

Context:
[Chunk 1 | label-1]
Do not apply more than 2.0 L/ha per application.

[Chunk 2 | memo-1]
The maximum application rate for Herbicide X is 2.0 L/ha.

Instructions:
- Base every claim on the context.
- Keep the answer concise.
```

One-line takeaway (interview-ready):

This prompt *forces grounding* by making the context the **only source of truth** for generation.

ok shouldn't the input_max chars, take into consideration the size of the entire prompt?

Yes — **100% correct**. 👍
That was a deliberate simplification in the reference code.

Short, precise answer (interview-ready):

Copy code

> **Yes.** The input budget must account for the **entire prompt**, not just retrieved context.

In a real system, input budget includes:

- System / role instructions
- Safety + grounding rules
- The user question
- Tool / formatting instructions
- **Retrieved context (chunks)** ← usually the largest part

Correct way to think about it

nginx

```
max_model_input
- prompt_overhead
- question_tokens
= available_context_budget
```

Why this matters

- Prevents **context truncation**

Copy code

- Avoids silent loss of instructions or citations
- Keeps model behavior predictable (critical in MCP systems)

Interview one-liner

> Context window management is about **budgeting tokens across the entire prompt**, not just retrieval output.

If you want, I can show the **exact corrected** `select_context()` **logic** that subtracts prompt + question size — that's a very strong follow-up in an LLM scientist interview.

can you give code for synthesizing answer using say bert model?

You *can't* really "synthesize" a new free-form answer with **BERT** because **BERT is encoder-only (not generative)**. What you *can* do with BERT is **extractive QA**: pick an answer span directly from the provided context (which is very compatible with "only use provided context").

Here's a clean, runnable example using a **BERT QA model** (SQuAD-finetuned) that answers **by extracting a span** from the retrieved chunks.

python

```python
# pip install transformers torch

from transformers import AutoTokenizer, AutoModelForQuestionAnswering
import torch

MODEL_NAME = "bert-large-uncased-whole-word-masking-finetuned-squad"

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModelForQuestionAnswering.from_pretrained(MODEL_NAME)
model.eval()

def bert_extractive_answer(question: str, context: str, max_len: int = 384
```

```
    """
    Extractive QA: returns a substring of `context`.
    Great for grounded RAG when you want to avoid hallucinations.
    """
    inputs = tokenizer(
        question,
        context,
        return_tensors="pt",
        truncation="only_second",   # truncate context if too long
        max_length=max_len,
        stride=64,
        return_overflowing_tokens=True,
        return_offsets_mapping=True,
        padding="max_length",
    )

    best = {"score": float("-inf"), "text": ""}

    # Handle sliding windows if context is long
    for i in range(inputs["input_ids"].shape[0]):
        input_ids = inputs["input_ids"][i].unsqueeze(0)
        attention_mask = inputs["attention_mask"][i].unsqueeze(0)
        offsets = inputs["offset_mapping"][i]  # (seq_len, 2)

        with torch.no_grad():
            out = model(input_ids=input_ids, attention_mask=attention_mask

        start_logits = out.start_logits[0]
        end_logits = out.end_logits[0]

        # Choose best span under a max answer length
        max_answer_len = 30
        for start_idx in torch.topk(start_logits, k=20).indices.tolist():
            for end_idx in torch.topk(end_logits, k=20).indices.tolist():
                if end_idx < start_idx or (end_idx - start_idx + 1) > max_a
                    continue

                start_char, end_char = offsets[start_idx].tolist()[0], offs
                if start_char == 0 and end_char == 0:
                    continue  # special tokens
```

```
                text = context[start_char:end_char].strip()
                score = start_logits[start_idx].item() + end_logits[end_id

                if score > best["score"] and text:
                    best = {"score": score, "text": text}

    return best["text"] if best["text"] else "I don't have enough context

# --- demo ---
context = (
    "Directions for Use: Do not apply more than 2.0 L/ha per application. "
    "Environmental Hazards: Maintain a 10 m buffer from surface water."
)
question = "What is the maximum application rate?"
print(bert_extractive_answer(question, context))
```

If you truly want "synthesis" (new phrasing), use a generative model

That's typically **T5/BART/GPT**. Example (T5) for *summarizing multiple chunks into one sentence*:

python

```
# pip install transformers torch

from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

MODEL_NAME = "t5-small"
tok = AutoTokenizer.from_pretrained(MODEL_NAME)
gen = AutoModelForSeq2SeqLM.from_pretrained(MODEL_NAME)

def t5_summarize_answer(question: str, context: str) -> str:
    prompt = (
        "Answer using only the context. If missing, say you don't have enou
        f"Question: {question}\n\nContext:\n{context}\n"
    )
    inputs = tok(prompt, return_tensors="pt", truncation=True, max_length=
    out = gen.generate(**inputs, max_new_tokens=80, do_sample=False)
```

```
    return tok.decode(out[0], skip_special_tokens=True)

print(t5_summarize_answer("What is the maximum application rate?", context
```

If you tell me which direction you want for the interview (extractive grounding vs generative synthesis), I'll tailor it to match the "agentic RAG + MCP" pipeline style.

i choose t5-small, thanks

Perfect choice 👍 — **T5-small** is a *very interview-friendly* pick.

Why T5-small works well here (short):

Copy code

- It's **encoder–decoder**, so it can *actually synthesize* answers.
- Fast, small, easy to explain.

Copy code

- Naturally fits **"text-to-text"** tasks like QA, summarization, and RAG.
- Much safer than pure LLMs when combined with a **constrained prompt**.

One-sentence interview framing

> *I used T5-small as a lightweight encoder–decoder model to synthesize grounded answers from retrieved context in a RAG pipeline.*

You're now well-aligned with:

Copy code

- **GenAI** → generative synthesis
- **RAG** → retrieval-grounded answers
- **Agentic AI** → deciding when/what to synthesize
- **MCP** → routing T5-small for low/medium-risk synthesis

If you want, next we can:

- plug T5-small **directly into the earlier pipeline**, or
- prep **exact interview talking points** for "why T5 vs GPT/BERT".

↳ AutoModelForSeq2SeqLM

in brief, explain why this is used

In brief:

`AutoModelForSeq2SeqLM` is used because it loads **encoder–decoder (sequence-to-sequence) models** like **T5**, which can **read input text and generate new output text**.

Why it's needed (one-liner):

> It enables **text-to-text generation** (QA, summarization, translation), which encoder-only models like BERT cannot do.

Key distinction (interview-safe):

- `AutoModelForSequenceClassification` → predicts labels
- `AutoModelForQuestionAnswering` → extracts spans
- `AutoModelForSeq2SeqLM` → **generates text**

That's why it's the correct choice for **RAG answer synthesis with T5**.

↳ truncation=True

explain

Got it — **very brief and precise** 👇

`truncation=True`
It tells the tokenizer to **automatically cut off tokens that exceed the model's maximum input length**, instead of throwing an error.

Why it's used:

- Prevents crashes when the prompt + context is too long

- Ensures input fits the model's fixed context window
- Essential in RAG where retrieved context size varies

Interview one-liner:

> Truncation guarantees the input respects the model's context limit, which is critical in retrieval-based pipelines.

then what is max_length

`max_length`

It explicitly sets the **maximum number of tokens** the tokenizer is allowed to produce for the input.

How it works with `truncation=True`:

- If tokenized input > `max_length` → it is **truncated to `max_length` tokens**
- If tokenized input ≤ `max_length` → nothing is cut

Why you need it:

- To control **exact prompt budget**
- To reserve space for generation (`max_new_tokens`)
- To avoid silently hitting model defaults

Interview one-liner:

> `max_length` defines the hard token budget for the input after which truncation is applied.

what if truncation is false

If `truncation=False`:

- The tokenizer **does not cut** the input.
- If the tokenized input **exceeds the model's max context length**, the model will **raise an error or fail** at runtime.

Why this is dangerous in RAG:

- Retrieved context size is variable
- Long prompts can crash inference
- Breaks reliability guarantees

Interview one-liner:

> Without truncation, overlong inputs cause runtime failures, which is unacceptable in production RAG systems.

↳ do_sample=False

in brief explain

`do_sample=False`

It makes generation **deterministic**: the model always picks the **highest-probability next token** (greedy/beam-style), instead of sampling.

Why it's used (especially in RAG):

- Produces **stable, repeatable answers**
- Reduces randomness and hallucinations
- Better for **factual, grounded** outputs

Interview one-liner:

> `do_sample=False` enforces deterministic generation, which is preferred for reliable, citation-based RAG answers.

```
for d in corpus:
    self.chunks.extend(chunk_text(d, chunk_size=chunk_size, overlap=overlap))
```

In brief:

This loop **preprocesses the entire document corpus into chunks** and stores them in one flat list.

What each part does:

- `for d in corpus:` → iterate over documents
- `chunk_text(d, ...)` → split one document into overlapping chunks
- `extend(...)` → add all chunks to `self.chunks` (not nested)

Why this is done:

- Retrieval works at the **chunk level**, not document level
- Smaller chunks improve recall and ranking accuracy
- Overlap preserves context across boundaries

Interview one-liner:

> We chunk documents upfront so retrieval operates on semantically meaningful, retrievable units rather than whole documents.

python extend function

```
list.extend(iterable)
```

It **adds each element** of the iterable to the list individually.

Example

python

```
a = [1, 2]
b = [3, 4]

a.extend(b)
# a → [1, 2, 3, 4]
```

Contrast with `append`

python

```
a = [1, 2]
a.append(b)
# a → [1, 2, [3, 4]]
```

Why `extend` here:

`chunk_text(...)` returns a **list of chunks**, and we want to add each chunk to `self.chunks`, not a nested list.

Copy code

One-liner:

> `extend` flattens an iterable into a list; `append` adds it as a single element.

nice explanation for doing overlap

Here's a **clean, intuitive explanation** you can use in interviews 👇

Why we use overlap when chunking

When you split a document into chunks, **important information can lie right at the boundary** between two chunks. Without overlap, that information gets **cut in half and loses meaning**.

Overlap ensures:

- Key sentences aren't split and lost
- Each chunk still makes sense on its own
- Retrieval can catch concepts that span chunk boundaries
- Better recall without needing much larger chunks

Intuition:

> Overlap is like repeating a few words at page breaks so readers don't lose the thread of the sentence.

Interview one-liner:

> Overlap preserves semantic continuity across chunk boundaries, improving retrieval accuracy with minimal extra cost.

Chunk(**{**ch.__dict__, "score": score}) explain

Nice line to ask about — it *looks fancy* but it's actually very practical.

What this does (in simple terms):
It creates a **new** `Chunk` **object** that is **identical to** `ch`, except the `score` field is replaced with a new value.

Break it down

python

```
ch.__dict__
```

→ gives all fields of `ch` as a dictionary

Copy code

python

```
{
  "doc_id": "...",
  "chunk_id": "...",
  "text": "...",
```

```
  "score": old_score,
  "metadata": {...}
}
```

python

```
{**ch.__dict__, "score": score}
```

→ copy all fields, then **override** `"score"` with the new value

Copy code

python

```
Chunk(** ...)
```

→ unpack that dictionary into the `Chunk` constructor

Copy code

Why not just do `ch.score = score`?

- Keeps `Chunk` objects **immutable-like** during scoring
- Avoids accidentally overwriting scores used elsewhere
- Makes retrieval/reranking **pure and side-effect free**

Interview one-liner

> This pattern clones the chunk while updating only the score, preserving immutability and avoiding side effects in retrieval pipelines.

q_tf = Counter(tokenize(query))

In brief:

This line converts the query text into a **term-frequency (TF) vector**.

What each part does

- `tokenize(query)` → splits the query into normalized words
- `Counter(...)` → counts how many times each word appears

Example

python

```
query = "maximum application rate"
q_tf = Counter(tokenize(query))
# {'maximum': 1, 'application': 1, 'rate': 1}
```

Copy code

Why it's used

- Enables **vector similarity** (cosine, dot product)
- Simple, fast baseline for retrieval
- Easy to explain and debug in interviews

Interview one-liner:

> This converts the query into a term-frequency vector so it can be compared numerically with document chunks.

scored.sort(key=lambda c: c.score, reverse=True)

In brief:

This line **sorts the list of chunks by their relevance score**, from highest to lowest.

Breakdown

- `scored.sort(...)` → sorts the list *in place*
- `key=lambda c: c.score` → sort using each chunk's `score`
- `reverse=True` → highest score first

Why it's done

- Retrieval produces many candidates
- Higher score = more relevant to the query
- Sorting lets you take the **top-K best chunks**

Interview one-liner:

This ranks retrieved chunks by relevance so downstream steps use the most relevant evidence first.

what is c the Chunk object?

Yes — `c` **is a** `Chunk` **object**.

In brief:

In `lambda c: c.score`, `c` **is each element of the** `scored` **list**, and each element is a `Chunk`.

Why this works

- `scored` is a list of `Chunk` objects
- Python passes each element to the lambda as `c`
- `c.score` accesses that chunk's relevance score

Equivalent expanded form

```python
def get_score(chunk):
    return chunk.score

scored.sort(key=get_score, reverse=True)
```

Interview one-liner:

`c` is just a loop variable representing each `Chunk` being sorted by its score.

Copy code

can you explain the rarity concept?

Sure — here's a **clear, intuitive explanation**.

Rarity concept (in reranking):
A term is *more informative* if it appears in **fewer chunks**. Matching on rare terms should matter **more** than matching on common ones.

Intuition

- Common words (e.g., *apply, product, use*) appear everywhere → weak signal
- Rare words (e.g., *buffer*, *L/ha*, *aquatic*) appear in few chunks → strong signal

How it's used

- Count in how many candidate chunks each query term appears (**document frequency**).
- Give **higher weight** to matches on terms that appear in fewer chunks.

Why this helps

- Improves precision in top results
- Prevents generic chunks from dominating
- Approximates the idea behind **IDF** (inverse document frequency)

Interview one-liner

> Rarity weighting boosts terms that are uncommon across chunks, because rare matches are more discriminative than common ones.

why not just compute idf?

Great question — this shows *real* IR understanding.

Why not compute full IDF here? (short answer)
Because this reranker is a **lightweight, local approximation**, not a full retrieval model.

Key reasons

- **Scope**: IDF is defined over the *entire corpus*; here we only rerank a **small candidate set**.
- **Cost**: Computing and storing global IDF requires preprocessing and updates when the corpus changes.
- **Simplicity**: Local rarity gives most of the benefit with far less complexity.
- **Latency**: Rerankers run in the hot path; lighter heuristics matter.

Conceptual framing

- Retriever → global statistics (BM25 / embeddings / IDF)
- Reranker → local, high-precision adjustments

Interview one-liner

> IDF is a global retrieval signal; local rarity is a cheap, sufficient approximation for reranking a small candidate set.

If you want, I can also give you the **perfect follow-up answer**: *"When would I switch to full IDF or BM25?"*

When would I switch to full IDF or BM25?

Short, interview-ready answer:

You switch to **full IDF / BM25** when retrieval quality depends on **global corpus statistics**, not just local reranking.

Specifically, use IDF / BM25 when:

- **Corpus is large or heterogeneous**
  (many products, regions, document types)
- **Recall matters**
  You can't afford to miss relevant chunks in the first retrieval stage
- **Vocabulary is skewed**
  Some terms are extremely common globally, others highly domain-specific

- **You need stability**
  Retrieval scores should be consistent across queries and time
- **You're building the *retriever***, not just the reranker

Keep local rarity when:

- You already have a strong retriever
- You're reranking a small candidate set
- Latency and simplicity matter

One-liner to remember

> **BM25 for recall, local rarity for precision.**

what is bm25

BM25 is a **classic information-retrieval ranking algorithm** that scores how relevant a document (or chunk) is to a query.

In simple terms:
It ranks text by combining:

- **Term frequency** → how often a query word appears in a document
- **Inverse document frequency (IDF)** → how rare that word is across the corpus
- **Length normalization** → avoids favoring very long documents

Why BM25 is used

- Strong baseline for text retrieval
- Works well without neural models
- Fast, interpretable, and robust
- Commonly used as the **first-stage retriever** in RAG systems

Interview one-liner:

> BM25 ranks documents using term frequency, global rarity (IDF), and length normalization to balance relevance and fairness.

defaultdictionary df.get()

Here's the **short, clear explanation** 👇

`defaultdict + df.get()`

- `defaultdict(int)` returns **0** for missing keys automatically.
- `df.get(key, 0)` safely retrieves a value, returning **0** if the key doesn't exist.

In your code

python

```python
df = defaultdict(int)
df.get(tok, 0)
```

Copy code

- If `tok` was never seen → returns `0`
- Prevents `KeyError`
- Makes rarity calculation safe

Why still use `.get()` with `defaultdict`?

- Defensive coding
- Clear intent: "this key may not exist"
- Works even if `df` later becomes a normal dict

Interview one-liner:

> `df.get(key, 0)` safely handles unseen terms and avoids errors when computing rarity or frequencies.

what does it actually do man?

😂 fair — here's the **plain, no-BS explanation**.

What `df.get(tok, 0)` actually does

It means:

> **"Give me the value for `tok` if it exists; otherwise give me 0."**

That's it. Nothing more.

Concrete example

python

```python
df = {"rate": 3, "buffer": 1}

df.get("rate", 0)      # → 3
df.get("gloves", 0)   # → 0   (because "gloves" is not in df)
```

Why this matters in your code

- You're counting **how many chunks contain a term**
- Some terms may **not appear at all**
- Instead of crashing, you safely treat them as having count `0`

Copy code

One-liner

> `df.get()` is a safe lookup that returns a default value when the key is missing.

```
for tok, qt in q_tf.items():
        if tok in ch_tokens:
            # fewer chunks contain tok => higher boost
            rarity = 1.0 / (1 + df.get(tok, 0))
            rarity_boost += 0.03 * rarity * qt
```

Alright, let's walk this **slowly and literally**, line by line — what it *actually* does.

python

```
for tok, qt in q_tf.items():
```

Copy code

Loop over **each query word**:

- `tok` = the word (token)
- `qt` = how many times it appears in the query (query term frequency)

---

python

```
if tok in ch_tokens:
```

Copy code

Only care if **this chunk actually contains that query word**.
No match → no contribution.

---

python

```
rarity = 1.0 / (1 + df.get(tok, 0))
```

Copy code

Compute **rarity** of the word:

- `df.get(tok, 0)` = number of chunks that contain this word
- More chunks → larger denominator → **smaller rarity**
- Fewer chunks → smaller denominator → **larger rarity**

👉 So **rare words are worth more**.

---

python

```
rarity_boost += 0.03 * rarity * qt
```

Copy code

Add a small score boost:

- `0.03` → scaling constant (keeps boost modest)

- `rarity` → reward rare terms
- `qt` → reward words repeated in the query

What this block does *in one sentence*

> For each query word that appears in a chunk, increase the chunk's score more if the word is rare and important in the query.

Ultra-short intuition

- Common word → tiny boost
- Rare word → bigger boost
- Word repeated in query → extra boost

Interview one-liner

> This is a lightweight IDF-style boost that favors chunks matching rare, query-important terms without computing full corpus statistics.

↳ `df.get(tok, 0)` = number of chunks that contain this word

this is not what you said earlier

You're right to call this out 👍 — let's **resolve the confusion cleanly**.

What I said earlier (correct, but implicit)

`df.get(tok, 0)` represents **how many chunks contain the token** `tok`
→ this is **document frequency**, but **computed locally** (only over candidate chunks).

What caused the confusion

The key subtlety is **where** `df` **comes from**.

Earlier in the code (important line):

python

```
df = defaultdict(int)
for tok in set(q_tokens):
    for ch in chunks:
        if tok in tokenize(ch.text):
            df[tok] += 1
```

So:

- `df[tok]` is **not global corpus frequency**
- It is **local frequency over the reranker's candidate set**

So the corrected, precise statement is:

Copy code

> `df.get(tok, 0)` = number of **candidate chunks** (not the full corpus) that contain `tok`.

Why this still matches the earlier explanation

- It is **rarity-based weighting**
- But it is **local rarity**, not global IDF
- That's why we said *"lightweight approximation, not full IDF"*

One-sentence reconciliation (interview-perfect)

> This code uses **local document frequency over candidate chunks**, which approximates IDF behavior without requiring global corpus statistics.

If you want, I can now show:

- the **exact mathematical difference** between this and BM25, or
- how to **upgrade this block to true IDF** with almost no extra code.

↳ It means: > **"Give me the value for `tok` if it exists; otherwise give me 0."**

what you said

Got it — let's be **very explicit** and clear the mismatch.

What I said earlier (about `df.get(tok, 0)`)

> **"Give me the value for `tok` if it exists; otherwise give me 0."**

That statement is **100% correct** — and it is about **Python behavior only**.

What the *code* is doing with that value (separate concept)

In *this specific code*, the value returned by `df.get(tok, 0)` **happens to mean**:

> "How many candidate chunks contain this token."

That meaning comes from **how `df` was constructed earlier**, not from `get()` itself.

The key distinction (this is the important part)

- `df.get(tok, 0)`
  - Pure Python operation
  - Safe dictionary lookup with default
- `df`
  - A dictionary you populated to count chunk occurrences
  - That's what gives the number its *semantic meaning*

So there is **no contradiction**:

- `get()` just fetches a value safely
- The *interpretation* of that value depends on what you stored in `df`

One-line reconciliation (interview-safe)

> `df.get(tok, 0)` is just a safe lookup; in this code, the retrieved value represents local document frequency because that's how `df` was built.

You were right to push on this — this is exactly the kind of precision interviewers like.

df is defaultdict from Collections

Yes — and that matters. Here's the **plain explanation**.

What `defaultdict(int)` actually is

python

```
from collections import defaultdict
df = defaultdict(int)
```

This means:

Copy code

- `df[key]` **automatically creates the key** if it doesn't exist
- The default value is `int()` → **0**

So concretely

python

```
df["rate"]        # → 0   (and now "rate" exists in df)
df["rate"] += 1   # → 1
```

Difference vs normal dict

Copy code

python

```
d = {}
d["rate"]        # ❌ KeyError
```

In your code, what `df` represents

Copy code

Because earlier you did:

python

```
df[tok] += 1
```

Copy code

👉 `df[tok]` ends up meaning:

"number of candidate chunks containing token `tok`"

Why `df.get(tok, 0)` still works

- With `defaultdict`, missing keys already default to `0`
- `.get(tok, 0)` just does the same thing **without creating the key**

Final one-liner (no ambiguity)

> `defaultdict(int)` ensures unseen tokens default to 0, allowing safe frequency counting without key errors.

You're asking exactly the *right* questions — this is deep IR + Python correctness territory.

in brief, say how and where i may use langgraph

In brief:
Use **LangGraph** to model the **agent's decision flow** in your RAG system as a **state machine**, instead of ad-hoc loops and `if` statements.

How you'd use it

- Define nodes like:
  - `classify_task`
  - `retrieve`
  - `rerank`
  - `rewrite_query`
  - `synthesize`
  - `policy_check`
- Define edges for decisions:
  - *If evidence insufficient → rewrite_query*
  - *If high risk → stronger model*
  - *If policy fails → stop / ask for clarification*

Where it fits

- **Agentic AI layer**, not retrieval itself
- Orchestration between tools, models, and policies
- Works cleanly with **MCP-style routing**

Interview one-liner

> LangGraph lets me express agentic RAG logic as an explicit, debuggable state graph rather than implicit control flow.

so give me some code, what would change, just small snippets, and what would happen to the mcp methods

Here are **small, focused snippets** showing what changes when you move from "manual loop + ifs" to **LangGraph state nodes + conditional edges**, and what happens to **MCP methods**.

---

1) Define the graph state (what flows between nodes)

python

```python
from typing import TypedDict, List, Optional

class GraphState(TypedDict, total=False):
    question: str
    task: str
    risk: str
    model_name: str
    retrieved: list
    reranked: list
    context: list
    draft: str
    ok: bool
```

```
    policy_reason: str
    retries: int
    trace: List[str]
```

What changed: instead of local variables, everything lives in `state`.

---

2) MCP becomes a node (or called inside nodes)

Option A: MCP is a dedicated node (cleanest)

python

```python
def route_with_mcp(state: GraphState, mcp) -> GraphState:
    spec = mcp.route(task=state["task"], risk=state["risk"])
    state["model_name"] = spec.name
    state["trace"].append(f"mcp.route -> {spec.name}")
    return state
```

Option B: MCP called inside `synthesize` node (also fine)

You'd call `mcp.route(...)` inside synthesize to choose which model to use.

Copy code

---

3) Nodes: retrieve → rerank → synthesize → policy check

python

```python
def retrieve_node(state: GraphState, tools) -> GraphState:
    state["retrieved"] = tools.retrieve(state["question"], top_n=12)
    state["trace"].append(f"retrieve -> {len(state['retrieved'])}")
    return state

def rerank_node(state: GraphState, tools) -> GraphState:
    state["reranked"] = tools.rerank(state["question"], state["retrieved"],
    state["trace"].append(f"rerank -> {len(state['reranked'])}")
    return state

def synthesize_node(state: GraphState, llm) -> GraphState:
    # llm could be T5-small, GPT, etc., picked using state["model_name"]
```

```
    state["draft"] = llm.generate(
        model_name=state["model_name"],
        question=state["question"],
        context=state["context"],
    )
    state["trace"].append("synthesize")
    return state

def policy_check_node(state: GraphState, mcp) -> GraphState:
    ok, reason = mcp.policy_check(state["draft"], state["context"])
    state["ok"] = ok
    state["policy_reason"] = reason
    state["trace"].append(f"policy_check -> {ok} ({reason})")
    return state
```

Copy code

---

4) Conditional edges replace `if` statements

You create a small “router” function that decides the next node.

python

Copy code

```
def need_retry(state: GraphState) -> str:
    # Example criteria: too little context or failed policy
    if len(state.get("context", [])) < 2 and state.get("retries", 0) < 1:
        return "rewrite"
    if state.get("ok") is False and state.get("retries", 0) < 1:
        return "rewrite"
    return "final"
```

Rewrite node (one retry):

python

Copy code

```
def rewrite_node(state: GraphState) -> GraphState:
    state["question"] = rewrite_query(state["question"])
    state["retries"] = state.get("retries", 0) + 1
```

```
    state["trace"].append("rewrite_query")
    return state
```

Context selection node (token/char budget happens here; budget could come from MCP route):

python Copy code

```python
def context_select_node(state: GraphState, budget_chars: int = 1800) -> Gr
    state["context"] = select_context(state["reranked"], max_chars=budget_
    state["trace"].append(f"context_select -> {len(state['context'])}")
    return state
```

---

5) Putting it together with LangGraph (minimal skeleton)

python Copy code

```python
from langgraph.graph import StateGraph, END

def build_graph(tools, mcp, llm):
    g = StateGraph(GraphState)

    g.add_node("classify", classify_node)           # sets task/risk
    g.add_node("mcp_route", lambda s: route_with_mcp(s, mcp))
    g.add_node("retrieve", lambda s: retrieve_node(s, tools))
    g.add_node("rerank", lambda s: rerank_node(s, tools))
    g.add_node("context", context_select_node)
    g.add_node("synthesize", lambda s: synthesize_node(s, llm))
    g.add_node("policy", lambda s: policy_check_node(s, mcp))
    g.add_node("rewrite", rewrite_node)

    g.set_entry_point("classify")
    g.add_edge("classify", "mcp_route")
    g.add_edge("mcp_route", "retrieve")
    g.add_edge("retrieve", "rerank")
    g.add_edge("rerank", "context")
    g.add_edge("context", "synthesize")
    g.add_edge("synthesize", "policy")
```

```
    # Conditional edge replaces: if insufficient/policy fail -> rewrite ->
    g.add_conditional_edges("policy", need_retry, {
        "rewrite": "rewrite",
        "final": END
    })

    g.add_edge("rewrite", "retrieve")

    return g.compile()
```

Invocation:

python Copy code

```
app = build_graph(tools, mcp, llm)
state = app.invoke({
    "question": "What is the max application rate and buffer zone?",
    "trace": [],
    "retries": 0,
})
print(state["draft"], state["ok"], state["trace"])
```

What changes to MCP methods?

Nothing fundamental. The MCP methods stay the same:

- `mcp.route(task, risk)`
  - called **once early** (recommended) to set `state["model_name"]` and budgets
- `mcp.policy_check(draft, context)`
  - called as a **gate node** near the end
- Optional: add `mcp.budget(prompt_overhead, question)`
  - then your `context_select_node` uses the returned budget

Key change: MCP decisions become **explicit nodes** (and therefore traceable), rather than hidden inside a big function.

If you want, I can adapt the earlier “reference implementation” into a compact LangGraph version while keeping it interview-sized (8–10 nodes, one retry).

ok just walk me through how these langgraph components are used, when mcp chooses a certain model

Here’s the flow, step-by-step, in plain terms—**what LangGraph is doing**, and **exactly when MCP picks a model**.

1) LangGraph core components, in this pipeline

State

A single dict-like object (`state`) that carries everything:

- question, task, risk
- retrieved chunks, reranked chunks, selected context
- chosen model name/spec
- draft answer, policy result
- trace + retry count

LangGraph passes this same `state` through every node.

Nodes

Each node is just a function:

- reads some keys from `state`
- writes new keys into `state`
- returns updated `state`

Example nodes: `classify` , `mcp_route` , `retrieve` , `rerank` , `context_select` , `synthesize` , `policy_check` , `rewrite` .

Edges

Edges define the order:

`classify → mcp_route → retrieve → rerank → context → synthesize →`

`policy_check`

Conditional edges

A router function decides what happens next based on the current state.

Example: after `policy_check`, either:

- go to `rewrite` (retry), or
- end (final)

---

2) When MCP chooses a model (the exact moment)

MCP chooses the model at the `mcp_route` node, right after classification.

Why right after classification?

Because classification produces the control signals MCP needs:

- `task` (lookup vs compare vs procedure)
- `risk` (medium/high)

Then MCP decides:

- which model to use (fast vs strong)
- budgets (max input/output)
- safety strictness

So the timeline is:

1. **classify node** sets:

python Copy code

```
state["task"] = "compare"
state["risk"] = "high"
```

2. **mcp_route node** runs:

python Copy code

```
spec = mcp.route(task=state["task"], risk=state["risk"])
state["model_name"] = spec.name
state["max_input_chars"] = spec.max_input_chars
```

3. From that point on, every downstream node respects those choices.

3) What changes downstream *because* MCP picked that model

A) Context selection uses MCP's budget

If MCP chooses a smaller/cheaper model → smaller context budget.

python Copy code

```
state["context"] = select_context(state["reranked"], max_chars=state["max_
```

So MCP affects **how many chunks you can fit**.

B) Synthesis uses MCP's model

When generating the answer, you use `state["model_name"]`:

python Copy code

```
draft = llm.generate(model_name=state["model_name"], question=..., context
```

So MCP affects **which model does the final generation**.

C) Policy check can be stricter for high-risk routes

Often, high-risk routes enforce stronger constraints:

- require more citations
- require higher coverage
- refuse if conflicting docs

That's `mcp.policy_check(...)`.

---

4) What happens if you retry (rewrite)

After policy check, conditional routing can send you to `rewrite`.

Important: you have two common designs:

Design 1 (simple): MCP routes once, keep same model

- You keep `state["model_name"]` unchanged
- Retry is only to improve retrieval

This is typical when you don't want model-flapping.

Design 2 (adaptive): retry can upgrade the model

If first pass fails, you call MCP **again** to escalate:

- First attempt: fast model
- If evidence is hard / policy fails: upgrade to strong model

In LangGraph, that just means your conditional edge goes:
`policy_check → mcp_route → retrieve` (instead of `policy_check → rewrite` only)

---

5) One compact "story" you can say in an interview

> "LangGraph runs my agent as a state machine. After classifying the question's task and risk, MCP routes to a model and sets budgets. Retrieval and context selection follow those budgets. The chosen model synthesizes the answer, then MCP policy-checks grounding. If policy fails or evidence is weak, the graph conditionally retries with a rewritten query—and optionally re-routes to a stronger model."

If you want, I can draw the graph as an ASCII diagram showing the exact nodes and conditional edges for both "route once" and "upgrade on failure."

↳ spec = mcp.route(task=state\["task"\], risk=state\["risk"\])

the if else for the choosing the model would still be in mcp, isn't langraph supposed to somehow take care of these decision points?

Yes — **the `if/else` can still live inside MCP**, and that's totally fine. LangGraph doesn't "magically remove" decisions; it **moves decisions to explicit, inspectable places**.

Think of it like this:

What LangGraph “takes care of”

LangGraph handles **control flow between steps**:

- *Which node runs next?*
- *Do we loop/retry?*
- *Do we branch (rewrite vs finalize vs escalate)?*
- *Do we stop?*

Those are **graph-level decisions** (edges).

What MCP is still responsible for

MCP handles **policy decisions about execution**:

- *Which model should be used?*
- *What budgets apply?*
- *What safety/grounding constraints apply?*

That’s **configuration/routing**, not workflow.

So yes, inside `mcp.route(...)` you’ll still have logic like:

- if high risk → stronger model
- else → cheaper model

The real benefit

LangGraph makes it so model choice is:

- **explicit in the trace** (you can see the routing decision as a step)
- **easy to change** (swap MCP implementation without touching the graph)
- **composable** (you can add an “escalate” branch in the graph)

If you want LangGraph to “own” the decision point

You can move the *branching* into LangGraph while keeping MCP as the source of truth.

Pattern: MCP returns a label, LangGraph branches

python Copy code

```
# MCP returns a route label, not a model directly
```

```
label = mcp.route_label(task, risk)  # "fast" or "strong"
```

Then LangGraph decides which node to run:

- `synthesize_fast`
- `synthesize_strong`

Graph-level routing is now in LangGraph, while MCP still provides the policy signal.

That looks like:

python Copy code

```python
def choose_synth_node(state):
    return state["route_label"]  # "fast" or "strong"
```

and conditional edges:

- `"fast" -> synthesize_fast`
- `"strong" -> synthesize_strong`

What changes in practice?

- The *if/else* moves from MCP into the **graph edge table**
- MCP becomes "policy oracle" that outputs the route label

Which is better?

- **Most production teams:** keep routing logic in MCP (centralized, consistent)
- **When LangGraph branching helps:** when routing affects *workflow*, e.g.
  - strong model also triggers extra steps (extra retrieval, stricter checks)

Interview one-liner:

> LangGraph manages *workflow branching*; MCP manages *model/policy routing*. You can integrate them by having MCP emit a route label that LangGraph uses to branch.

If you want, I can show a tiny 10-line example of the "route label + conditional edge" style so it's crystal clear.